

`datagrid/filtering.php` and explore!

There is a whole lot more to codegen; we will highlight more features in the articles to come.

HTML templates

QCubed gives you room to keep your 'control' and 'presentation' issues separate. You define how your page 'behaves' in one file, and how it looks in another. There's little need to have HTML mixed with PHP code. Will it allow you to work faster? You can change the look (styles and structure) of a page easily by using a different 'template' file for each condition. Developing a mobile site is not going to take a lot of extra work—there will be no need to write the logic twice.

jQuery integration

Too well known to require an introduction, jQuery is the most famous JavaScript library out there, and you have tons of plug-ins. jQuery comes built into QCubed. In fact, QCubed even allows you to create jQuery UI objects using PHP. The JavaScript code is automatically created for you when you visit (render) the page. QCubed uses jQuery to get AJAX done so easily, yet it produces jaw-dropping effects.

AJAX and event-driven programming

With QCubed, you seize all control over defining how your page will behave on different occasions. Define an action on any control (individual HTML elements such as buttons, text-boxes, divs, spans, text, images, etc) in QCubed (using PHP, of course) for events like a click, mouse hover, keypress, etc—in short, for all the events JavaScript can detect. Then write an event-handler for the action (in PHP, again) and let the rest be lost in oblivion—QCubed will take over. Remember that when you create an event and its handler, you have to specify the type of action it is. There are three at your disposal:

- **QJavaScriptAction**—You write a function in a JavaScript file, include that in your page, and tell QCubed that you want to execute the same at the occurrence of the event. You are not even writing jQuery code. QCubed—at your service!
- **QServerAction**—The page will reload itself with all the changes that you commanded in the event-handler function. There's no need to worry about POST data, creating a long URL string or handling the GET parameters.
- **QAjaxAction**—An AJAX call is sent to the server, the event-handler function is executed and the server sends back the changes. Yes, AJAX can be activated by changing the action type! Forget about writing separate functions to test what works with AJAX and what doesn't. And you do not need to create a new PHP page to handle a complicated query-string; you just need to write functions.

The event-driven architecture of QCubed is so flexible that it requires another article for me to share all the details.

Caching

If you have *memcached* running on your server, mention that to QCubed in the configuration file, and enable caching on the database you want. All your 'Load' function calls start utilising *memcached* to store the PHP objects of your DB table rows. Once again, there is no need to write code to handle caching. You only need to activate the functionality.

Security

Security is an inevitable aspect for any Web application, more so in PHP, since it's easier to remain 'insecure' while using PHP than other languages—but it is still fast and easy enough to lure a huge number of developers from all over the world. QCubed closes some holes that are left behind by PHP.

When you are using QCubed's query system, the SQL statements generated are properly escaped before being sent to the server, to prevent SQL injection attacks. QCubed also comes with the *HTMLPurifier* library, which can be used to filter HTML code coming from text-boxes, and to filter them as per your requirements. The default values prevent XSS attacks well enough, though you can customise the filtering mechanism.

It is a little dangerous to keep your *includes* directory in your *DocumentRoot*. QCubed allows you to move the *includes* directory elsewhere by editing just one line in each of two files. You are not going to need to modify your entire code-base. In addition, QCubed allows you to take control of the execution of sensitive scripts. Let's say you have a script that updates your database or generates reports, which you want accessible to you alone. You can use QCubed to ensure that only IP addresses defined in its configuration file can access those pages. This functionality too is just a function call away.

Extensibility

QCubed is fully OOPS, and thus very extensible. There are many good plug-ins and you can always enhance the functionality of your project. The best thing about plug-ins is the 'plug-in installer'. You can actually divide your full application into smaller plug-ins and make installable packages. Welcome to modularity!

So that was just the tip of the iceberg. There is a whole lot more you can do with QCubed... but wait, one of the best things about it is its licence. QCubed is available under the MIT licence, which means you can use it in another OSS project, or even a commercial one. No one will come after you with lawyers or a court summon. And before we call it a day, let me tell you that it's free of cost as well. We shall explore more about QCubed in the articles to come. 

By: Vaibhav Kaushal

The author is a 25 year old college dropout from Bengaluru, who also happens to be a core contributor to QCubed. He loves writing for technology magazines and his blog when he is not busy fiddling around with QCubed or developing his website (<http://www.c-integration.com/>).